Design, Deployment and Cloud-Native Enhancement of a Three-Tier Web Application on AWS

OccasionHub: event-venue booking platform

SE-490 Cloud Computing · Complex Engineering Activity · Department of Software Engineering
Ali Afzal (SE-23048)

Source code: `https://github.com/Woodenbowl000/occasionhub-cloud`

November 8, 2026

---

1 Introduction and choice of cloud provider

OccasionHub is our venue-booking web application (search venues by city, type, price and distance; negotiate and book; admin approval), previously hosted on Vercel and Neon. We containerised it and deployed it on **AWS** (`ap-southeast-2`, Sydney) as three tiers: an Nginx + Next.js 15 *web tier*, an Express + Prisma *application tier* (Node.js 22), and an Amazon RDS PostgreSQL 16 *database tier*. Terraform defines the environment and GitHub Actions releases it.

Why AWS. Every requirement maps to a mature native service (VPC, ALB, EC2 Auto Scaling, RDS, ECR, Secrets Manager, CloudWatch). RDS PostgreSQL supports *PostGIS*, which the app needs for venue coordinates and distance search. GitHub Actions has first-party AWS actions for credentials and ECR. The Terraform AWS provider covers every resource, so the stack is created and destroyed with one command. Free Tier credits cover the t3.micro and db.t4g.micro instances.

2 Overall architecture and design decisions

Figure 1 shows the deployed system. A user's request enters through the Internet Gateway to the internet-facing ALB, which forwards it to a web-tier instance. Nginx serves the Next.js pages and forwards every `/backend/*` call to an *internal* ALB. The internal ALB balances requests over the application instances, which query RDS over TLS. Nothing behind the public ALB has a public IP address.

Network design. One VPC (`10.0.0.0/16`) spans two Availability Zones (AZs). Each AZ has four /24 subnets, one per layer, so every tier is isolated by routing as well as by firewall rules (Table 1).

Table 1: Subnets and routing (identical layout in AZ A and AZ B).

Layer	CIDR (AZ A / AZ B)	Default route	Contents
Public	10.0.1.0/24, 10.0.2.0/24	Internet Gateway	internet-facing ALB, NAT gateway
Web (private)	10.0.11.0/24, 10.0.12.0/24	NAT gateway; S3 via gateway endpoint	web-tier EC2 instances (ASG)
App (private)	10.0.21.0/24, 10.0.22.0/24	NAT gateway; S3 via gateway endpoint	internal ALB, app-tier EC2 instances (ASG)
DB (isolated)	10.0.31.0/24, 10.0.32.0/24	none (VPC-local only)	RDS PostgreSQL (and optional standby)

Key design decisions.

- **Two load balancers:** the API sits behind an *internal* ALB, so it is never reachable from the internet, and it gets its own target group, health checks and scaling.
- **Same-origin API:** the browser calls `/backend/...` and Nginx proxies it. There is no CORS on the request path and no API hostname baked into the bundle, so the same image runs locally and in the cloud.
- **Stateless instances:** photos go to S3 and sessions are JWTs, so any instance can be added, replaced or removed at any time.
- **Immutable releases:** images are tagged with the git SHA, and the SSM parameter `/occasionhub/image-tag` names the live release. A deploy means "change the pointer, replace instances".
- **EC2 + Docker, not Kubernetes:** the simplest platform that meets every requirement on Free Tier-sized instances; the containers can move to ECS or EKS unchanged.

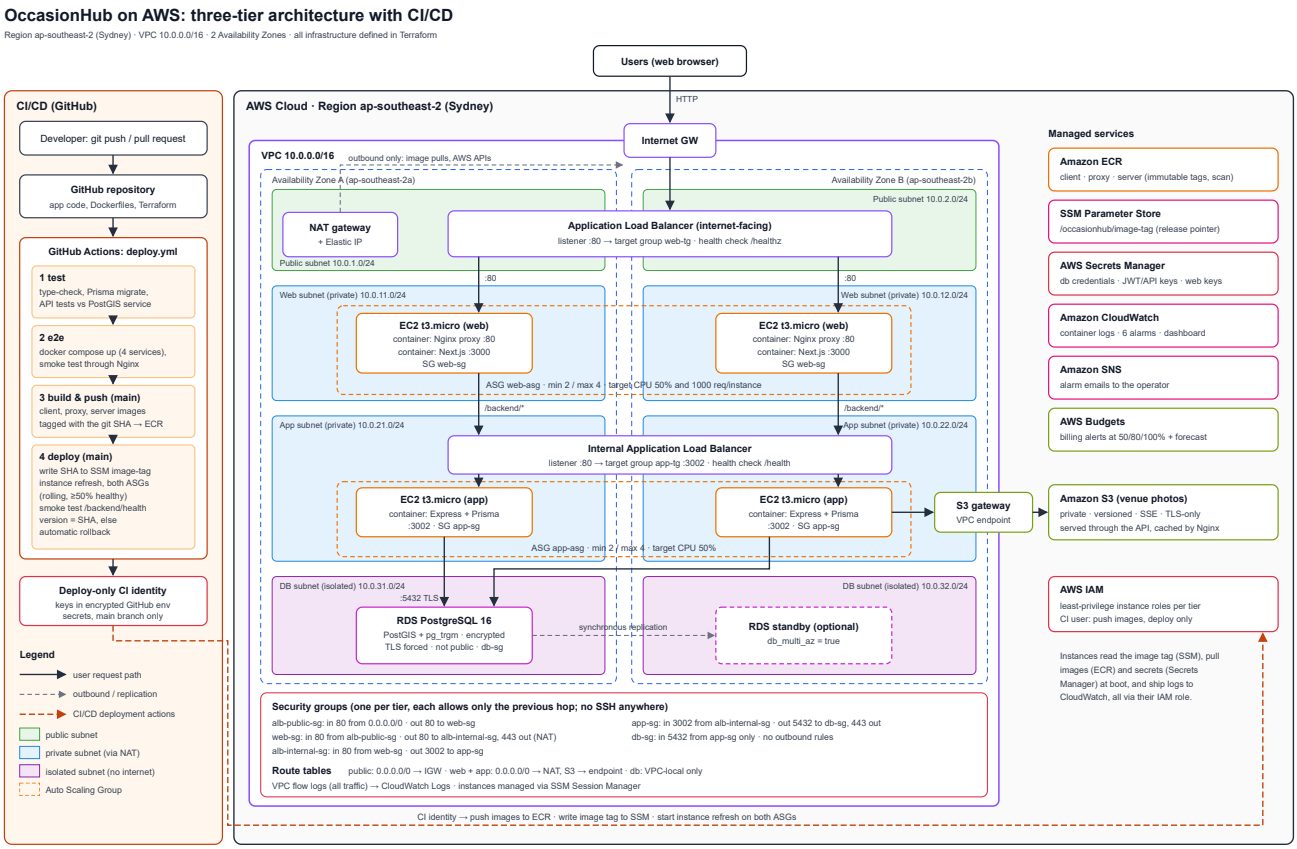


Figure 1: Deployed architecture: network layers, both Availability Zones, load balancers, Auto Scaling Groups, managed services, security groups and the CI/CD flow (source: `docs/architecture.svg`).

3 Services used and their role in each tier

Table 2: AWS services and their role.

Service	Role in the solution
VPC, subnets, IGW, NAT gateway, route tables	Network layer described in Table 1; NAT gives private instances outbound-only access (image pulls, AWS APIs).
Application Load Balancer ×2	Public ALB → web target group (port 80, health check `/healthz`); internal ALB → app target group (port 3002, `/health`).
EC2 + Auto Scaling Groups	Amazon Linux 2023 t3.micro instances from launch templates; web and app ASGs (min 2, max 4) across both AZs with target-tracking policies.
Amazon RDS for PostgreSQL 16	Managed database in the isolated subnets: encrypted gp3 storage, automated backups, forced TLS, slow-query log; optional Multi-AZ standby.
Amazon ECR	Private registry, one repository per image (`client`, `proxy`, `server`); immutable tags, scan on push, keep the last 10 images.
AWS Secrets Manager	Database credentials, JWT signing key and optional Stripe, Google and Resend keys; read at container start through the instance role.
SSM Parameter Store	Release pointer `/occasionhub/image-tag` written by the pipeline.
Amazon S3 + gateway endpoint	Private, versioned, encrypted bucket for venue photos; reached without the NAT through a free gateway endpoint.
CloudWatch, SNS	Container and VPC flow logs, 6 alarms, one dashboard; alarm and recovery emails through SNS.
AWS Budgets	Billing alerts at 50%, 80% and 100% of a monthly limit, plus a forecast alert.
IAM	Least-privilege role per tier; a deploy-only CI identity for GitHub Actions. Instances are managed with SSM Session Manager; no SSH.

4 Containerisation and CI/CD pipeline

Images. Each tier has a multi-stage Dockerfile. The final images are slim, run as non-root users and define a `HEALTHCHECK`.

- `server/`: compiles TypeScript and the Prisma client, then ships only production dependencies, with `tini` for graceful shutdown. Its entrypoint loads secrets, runs `prisma migrate deploy` (safe from many instances because it takes a lock), optionally seeds data, then starts the API.
- `client/`: Next.js *standalone* output; the runtime holds 56 MB instead of 643 MB of `node_modules`.
- `proxy/`: `nginx-unprivileged`, with its configuration rendered from environment variables at start.

`docker-compose.yml` runs db, server, client and proxy on three networks (`web`, `app`, `data`) that mirror the cloud segmentation.

Pipeline (`.github/workflows/deploy.yml`):

1. **test** (every push and pull request): type-check, migrate and seed a PostGIS service container, then run 11 API tests.
2. **e2e**: `docker compose up -build -wait`, then smoke-test the pages, the API and venue search through Nginx.
3. **build-push** (`main` only): a matrix job pushes the three images to ECR, tagged with the SHA and built with the layer cache.
4. **deploy** (`main`, environment `production`): `scripts/deploy.sh` writes the SHA to the release pointer and starts an *instance refresh* on both ASGs, so new instances must pass ALB health checks while at least 50% keep serving. It then checks that `/backend/health` reports the new SHA. On failure it restores the previous tag and refreshes again (automatic rollback).

5 Security considerations

- **Security groups:** one per tier, each admitting only the previous hop by security-group reference: internet → public ALB :80 → web :80 → internal ALB :80 → app :3002 → database :5432. Egress is explicit; the database has none. No SSH anywhere; operators use SSM Session Manager.
- **Segmentation:** only the ALB and NAT are public. Web and app instances have no public IPs. Database subnets have no internet route, and RDS is not publicly accessible. VPC flow logs record all traffic.
- **Secrets:** Terraform generates the database password and JWT key into Secrets Manager. Containers read them at start through the instance role, so nothing secret is in images, launch templates, user data or git. Locally, values come from a git-ignored `.env`.
- **Pipeline:** keyless OIDC federation was the first choice, but the AWS-managed service control policy of our new-experience (Free plan) project denies `iam:*Provider*`, so no identity provider can be created. The pipeline therefore uses a dedicated IAM user with no console password and one inline policy: push to three ECR repositories, write one parameter, refresh `occasionhub-*` ASGs. Its keys are encrypted GitHub *environment* secrets of `production`, which only the `main` branch can use, and are masked in logs. The secret exists only in the Terraform state, which is kept outside the project folder, and is rotated with one `terraform apply -replace`. Setting `github_auth_mode = "oidc"` restores keyless login on an account that allows it.
- **Least-privilege roles:** the web role reads only its images, the release pointer and the web secret. The app role adds the database and app secrets and `venues/*` in the photo bucket.
- **Encryption and hardening:** at rest on RDS, EBS, S3 and ECR. TLS is forced to RDS (`rds.force_ssl=1`) and S3 (deny non-TLS policy). Instances require IMDSv2, containers run as non-root, ALBs drop invalid headers, Nginx adds security headers, and images are scanned on push.
- **Application fixes:** venue sorting now accepts only a fixed set of columns, page size is capped at 100, and photo names are validated before being read from S3.

6 Cloud-native enhancements

Five enhancements were implemented:

- **Infrastructure as code:** about 1,700 lines of Terraform, validated.
- **Object storage:** S3 for venue photos.
- **Persistent block storage:** encrypted gp3 EBS volumes for instances and RDS (storage auto-scaling is one variable away; off by default to stay within the Free Tier's 20 GB).
- **Centralised logging and monitoring:** CloudWatch.
- **Cost estimate and optimisation:** Tables 3 and 4.

Monitoring. Docker's `awslogs` driver streams every container to CloudWatch Logs, and RDS exports statements

slower than 500 ms. Six alarms email through SNS: ALB 5xx errors, p95 latency above 1 s, unhealthy hosts per tier, database CPU and database storage. A dashboard shows traffic, latency, CPU per tier, instances in service, healthy targets and recent API errors.

Cost estimate. Table 3 uses AWS on-demand prices for `ap-southeast-2` (Sydney), 730 hours a month and light traffic. Unit prices were retrieved from the AWS Price List API (`aws pricing get-products`) on November 8, 2026.

Table 3: Estimated monthly cost (USD), default configuration.

Item	USD / month
4 × EC2 t3.micro (2 web + 2 app), $0.0132/h	38.54
4 × 20 GB gp3 EBS ($0.096/GB)	7.68
2 × Application Load Balancer ($0.0252/h + about 0.5 LCU each)	42.63
NAT gateway ($0.059/h + 10 GB processed)	43.66
3 public IPv4 addresses (2 on the ALB, 1 on the NAT)	10.95
RDS db.t4g.micro single-AZ ($0.025/h) + 20 GB gp3 ($0.138/GB)	21.01
Secrets Manager (3 secrets), CloudWatch logs, S3, ECR	4.20
Total, about $0.23 per hour	**168.67**

Table 4: Cost optimisation measures.

Measure	Saving / month	Trade-off
Run only while needed: deploy, collect evidence, `terraform destroy` (implemented: whole stack is code; billing alerts via AWS Budgets)	up to 100%	recreation takes about 15 min
1-year Compute Savings Plan + RDS reserved instance	about $20 (30–40% of EC2/RDS)	1-year commitment
Graviton (t4g.micro) with multi-arch images (buildx)	about $8 (20% of EC2)	arm64 build stage in CI
Scheduled scaling to 1 instance per tier at night (ASG scheduled actions)	about $10	no redundancy during those hours
NAT instance (t4g.nano) instead of NAT gateway, dev only	about $39	self-managed, lower throughput

Application performance. Moving to the cloud exposed four inefficiencies, now fixed:

- **N+1 query:** venue search ran one PostGIS query per venue (24 sequential database round trips for 20 venues); it now uses one batched query plus a parallel count (5 round trips).
- **Missing indexes:** a migration adds a trigram index for city search, a GIST spatial index and three B-tree indexes.
- **Payload size:** the API gzips JSON, and lazy-loading the Mapbox map cut `/venues` first-load JavaScript from 676 kB to 237 kB.
- **Edge caching:** Nginx compresses text and caches content-hashed assets and photos.

7 Testing results

Local verification (November 8, 2026). All 11 API tests pass against PostgreSQL + PostGIS: liveness and readiness, venues with coordinates, gzip, page-size cap and sort whitelist, distance order, case-insensitive city search, admin sign-in returns a JWT, and wrong password rejected. Also confirmed:

- migrations apply cleanly and the seed script is idempotent;
- `EXPLAIN` shows city search using the trigram index;
- `terraform validate` succeeds and the rendered user-data scripts pass `bash -n`;
- Nginx passed `nginx -t` and a live run in front of the built front end and API: `/backend` routing, gzip, single security headers, and static-asset cache `MISS`→`HIT`.

Table 5: Venue search, before and after optimisation (1,000 requests, concurrency 10, 20 venues per page).

Variant	Requests/s	p50 latency (ms)	p95 latency (ms)
Before (N+1 coordinate queries)	108–218	46–88	60–129
After (batched query, parallel count)	391–410	23–24	33–36

Automated deployment on AWS (8 Nov 2026). Pushing commit `e7f505a`, a visible change to the landing-page tagline, to `main` triggered pipeline run #4 (Table 6). The first deploy attempt stopped at its pre-flight check after 12 s because the `APP_URL` repository variable was not yet set. No AWS resource was changed, so the safety check worked as designed. After the variable was added, re-running the failed job replaced all four instances with the new release while the site stayed up. `/backend/health` then reported `e7f505a` at `http://occasionhub-public-alb-777768551.ap-southeast-2.elb.amazonaws.com` (taken down after evaluation to save cost). Of 24 consecutive requests, the two web instances answered 12 and 12 (`X-Served-By`) and the two app instances 13 and 11 (`X-App-Instance`), which confirms load balancing in both tiers (Figures 10–11).

Table 6: Pipeline run #4 for commit `e7f505a` (GitHub Actions).

Job	Result	Duration (m:ss)
Test API against PostgreSQL + PostGIS	passed	0:43
Build all images + end-to-end smoke test (docker compose)	passed	2:06
Build & push server / client / proxy (in parallel)	passed	0:28 / 1:33 / 0:31
Deploy: release pointer, rolling refresh of both ASGs, smoke test	passed	6:54

8 Non-functional requirements

Table 7: How the design meets each NFR, and what it does not yet address.

NFR	Design decisions that support it	Known limitations / improvement
Scalability	Stateless containers (JWT sessions, photos in S3); separate ASGs and target groups per tier scale independently up to 4 instances each; batched queries and indexes reduce per-request database load.	Single RDS instance limits write scale; add read replicas, RDS Proxy for connection pooling, and Redis for caching.
Availability / Reliability	Two AZs for every layer; at least 2 instances per tier; ALB health checks and ELB-type ASG health checks replace failed instances automatically; rolling deploys keep at least 50% serving; automatic rollback; daily RDS backups.	RDS single-AZ and a single NAT gateway by default (cost). Set `db_multi_az = true` and `single_nat_gateway = false` for production.
Elasticity	Target tracking on CPU (50%) for both tiers and on ALB requests per target for the web tier, so capacity grows and shrinks automatically between min and max; RDS storage auto-scaling can be enabled (`db_max_allocated_storage`).	New instances take about 2–3 minutes (package install and image pull); a pre-baked AMI or ECS on Fargate would react faster.
Security	See Section 5: per-tier security groups, isolated database subnets, Secrets Manager, branch-restricted deploy-only CI identity, least-privilege IAM, encryption, IMDSv2, non-root containers, flow logs.	HTTP only: no domain or certificate (add ACM + HTTPS listener and AWS WAF). CI uses long-lived keys because the project SCP blocks OIDC: rotate them, or switch to OIDC on an advanced account.
Maintainability / Deployability	One command builds the infrastructure; every push is tested, built, deployed and verified without manual steps; immutable SHA-tagged images make each release traceable and reversible; identical containers locally and in the cloud.	Terraform runs manually with local state; move state to S3 with locking and add a plan/apply pipeline.
Performance / Latency	Nginx gzip and caching; static assets cached for a year; smaller JavaScript bundle; N+1 query removed (median latency 46–88 ms to about 24 ms); keep-alive tuned above the ALB idle timeout; same-region placement of all tiers.	No CDN; Amazon CloudFront in front of the ALB would cut latency for distant users.
Cost-Efficiency	Free Tier-sized instances; scaling to the minimum when idle; S3 gateway endpoint avoids NAT data charges; 14-day log retention; ECR lifecycle policy; billing alerts; whole stack destroyable (Table 4).	Fixed hourly costs (2 ALBs, NAT) dominate at low traffic; a single-host deployment would cost far less for a demo.
Portability	Every tier is a standard OCI container; configuration only through environment variables; the same images run with docker compose, on EC2, or on ECS, EKS or another cloud's VMs; PostgreSQL is open source.	AWS-specific glue (Secrets Manager loader, S3 client, Terraform) would need a provider swap; Kubernetes manifests would ease that move.

9 Conclusion

OccasionHub now runs on AWS as a segmented, horizontally scalable three-tier system. Infrastructure, images and releases are all reproducible from the git repository. A push to `main` tests the code, builds and publishes three images, rolls them out across two Availability Zones without downtime, and verifies the result. The migration also made the application measurably faster.

The main gaps are HTTPS, a Multi-AZ database and remote Terraform state. Each is a configuration change, not a redesign.

Appendix A: Deployment evidence

All figures were captured on 8 Nov 2026 from the live deployment in the AWS Management Console (`ap-southeast-2`), the public GitHub Actions run and the live site.

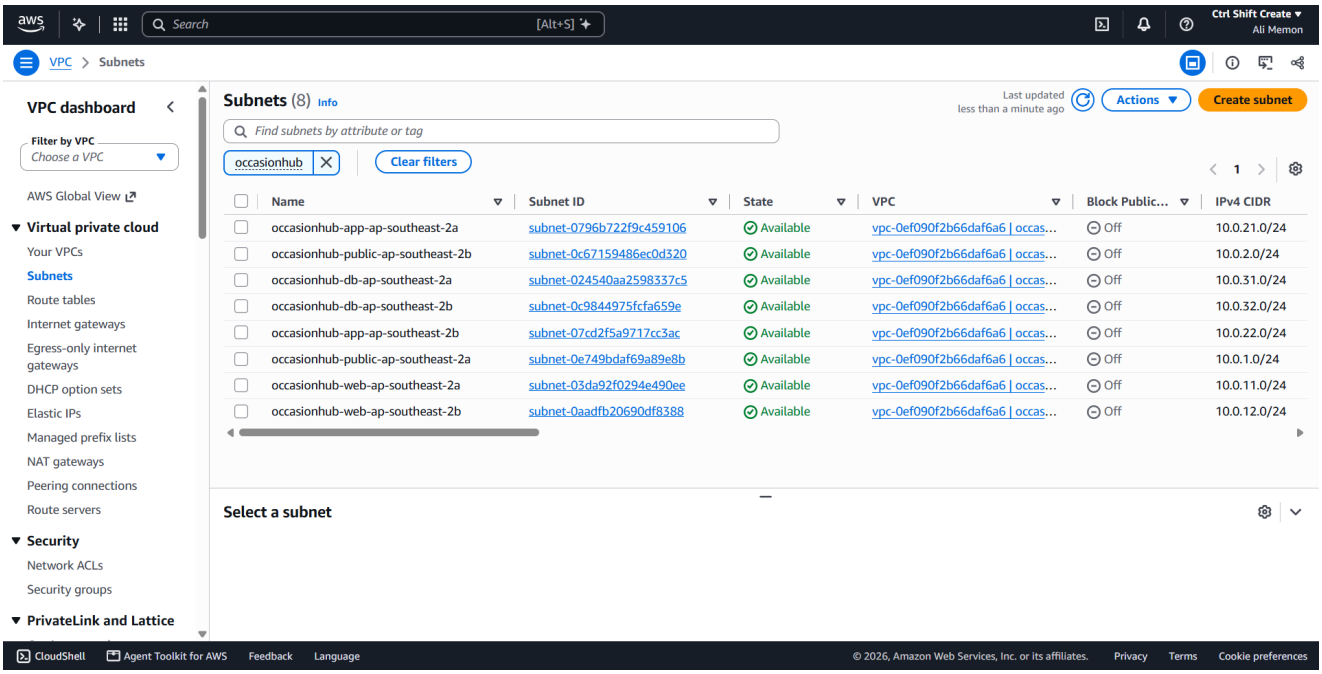


Figure 2: VPC `occasionhub-vpc` with its eight subnets across two AZs.

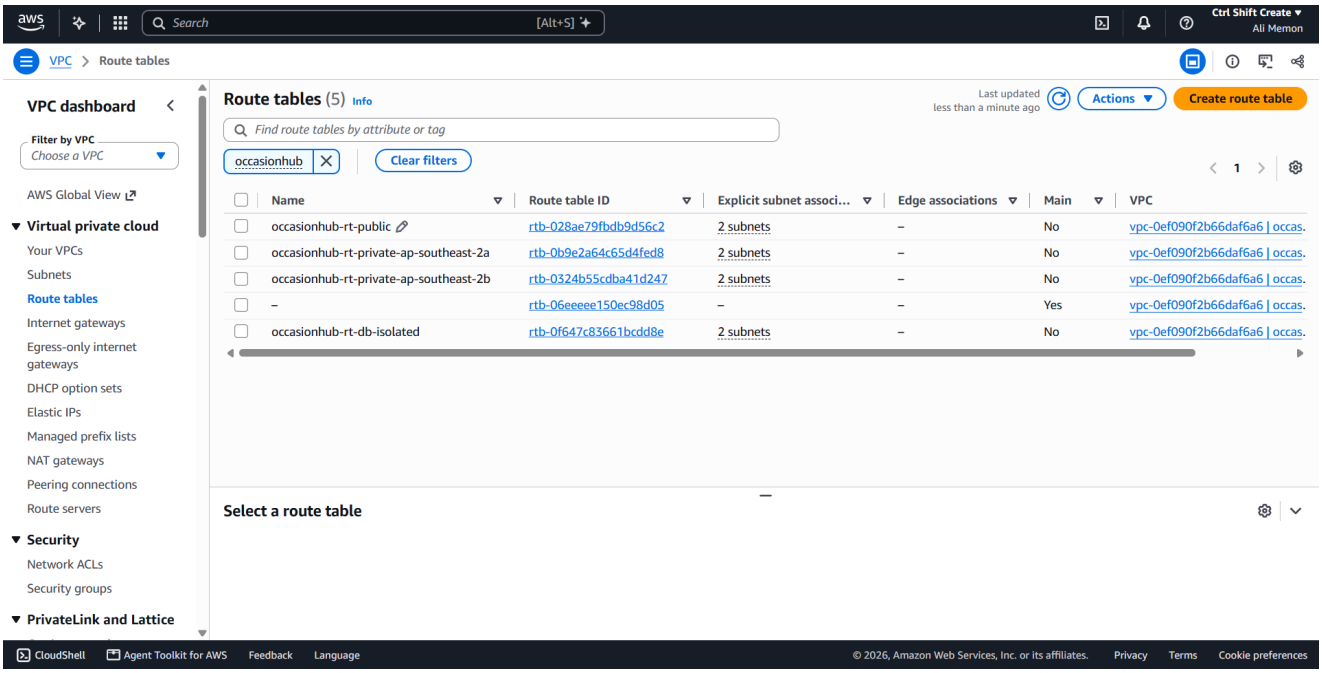


Figure 3: Route tables: one public, one private per AZ and one isolated database table, each associated with two subnets.

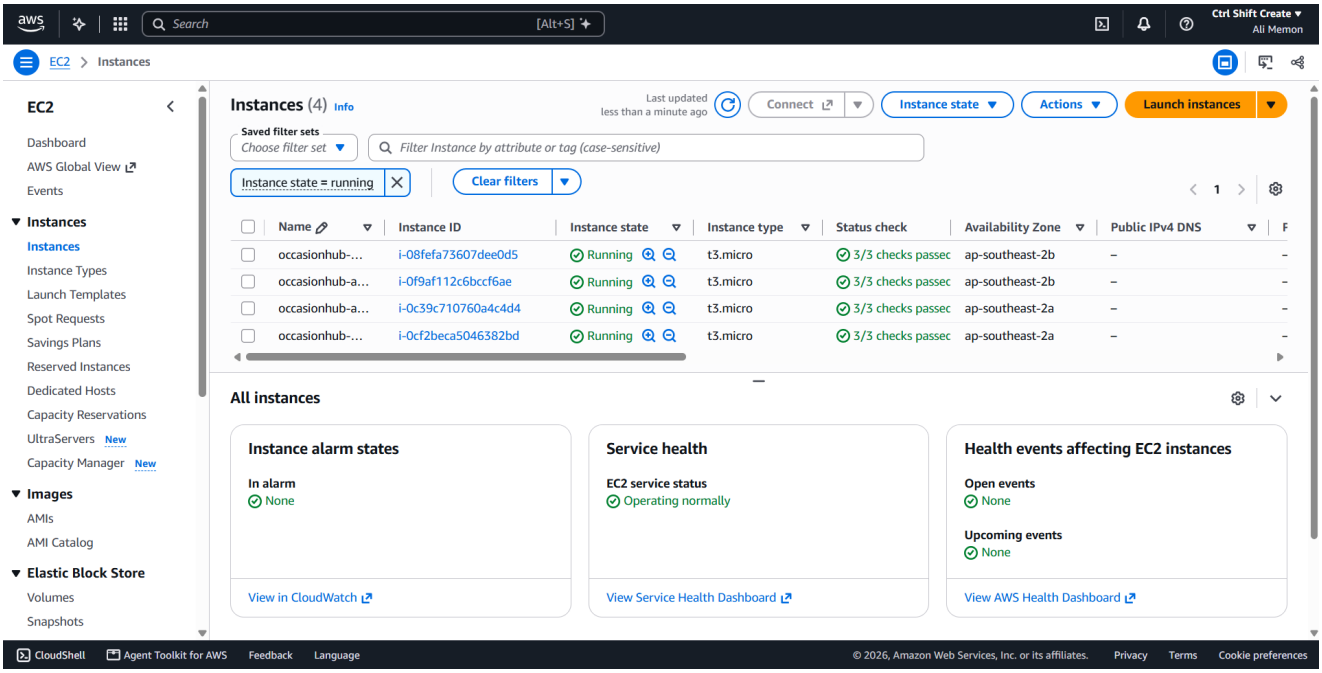


Figure 4: EC2 instances: two web and two app instances running in two AZs, all status checks passed, no public IPv4 address.

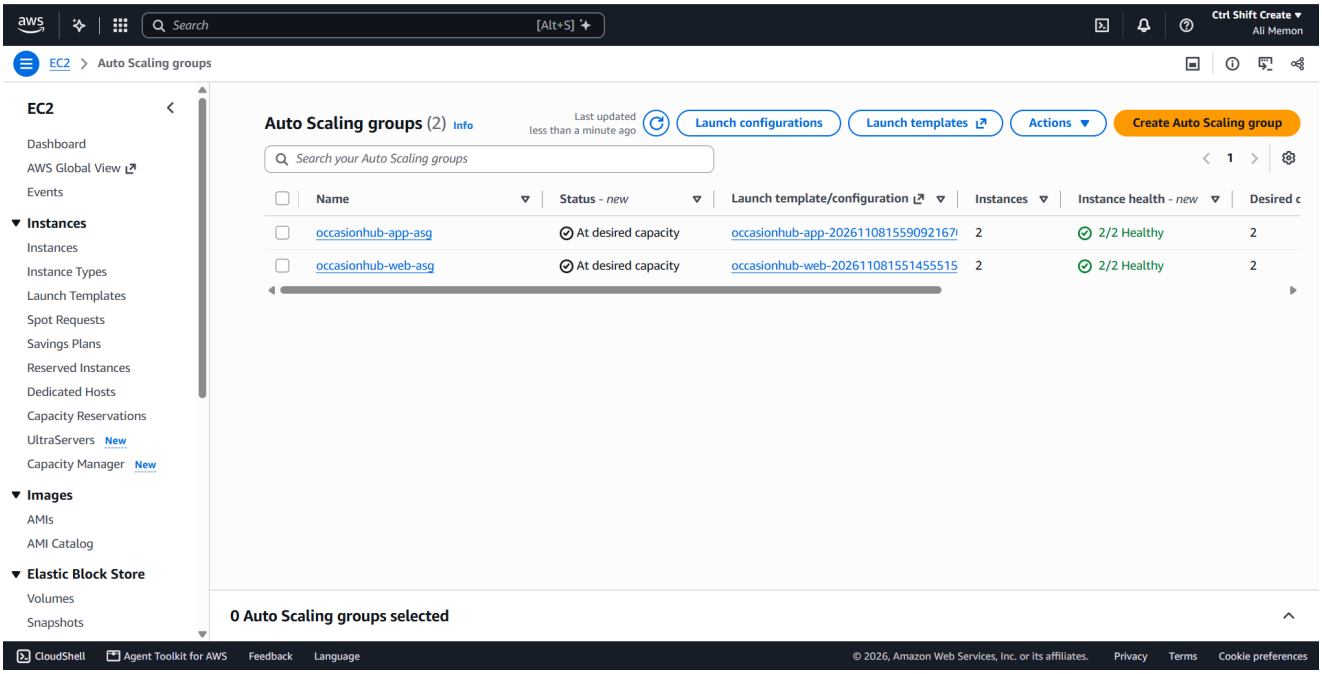


Figure 5: Auto Scaling Groups `occasionhub-web-asg` and `occasionhub-app-asg` with target-tracking policies.

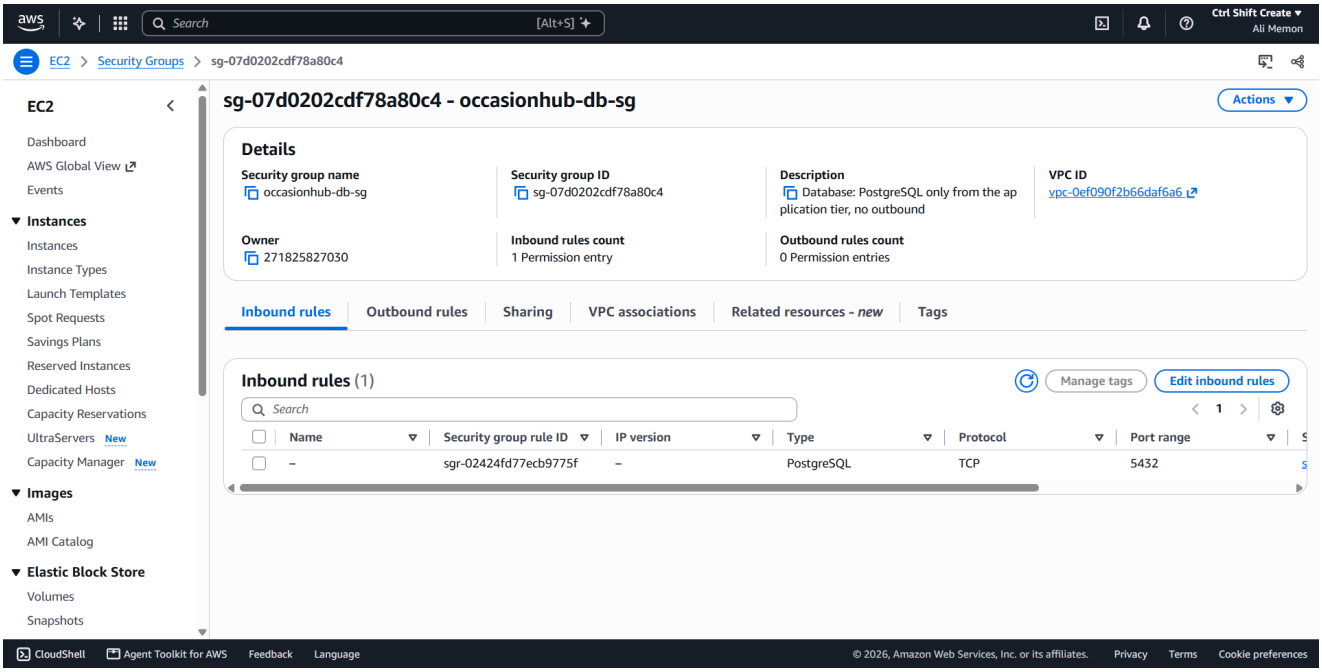


Figure 6: Security group `occasionhub-db-sg`: one inbound rule (PostgreSQL 5432, from the application tier only) and no outbound rules.

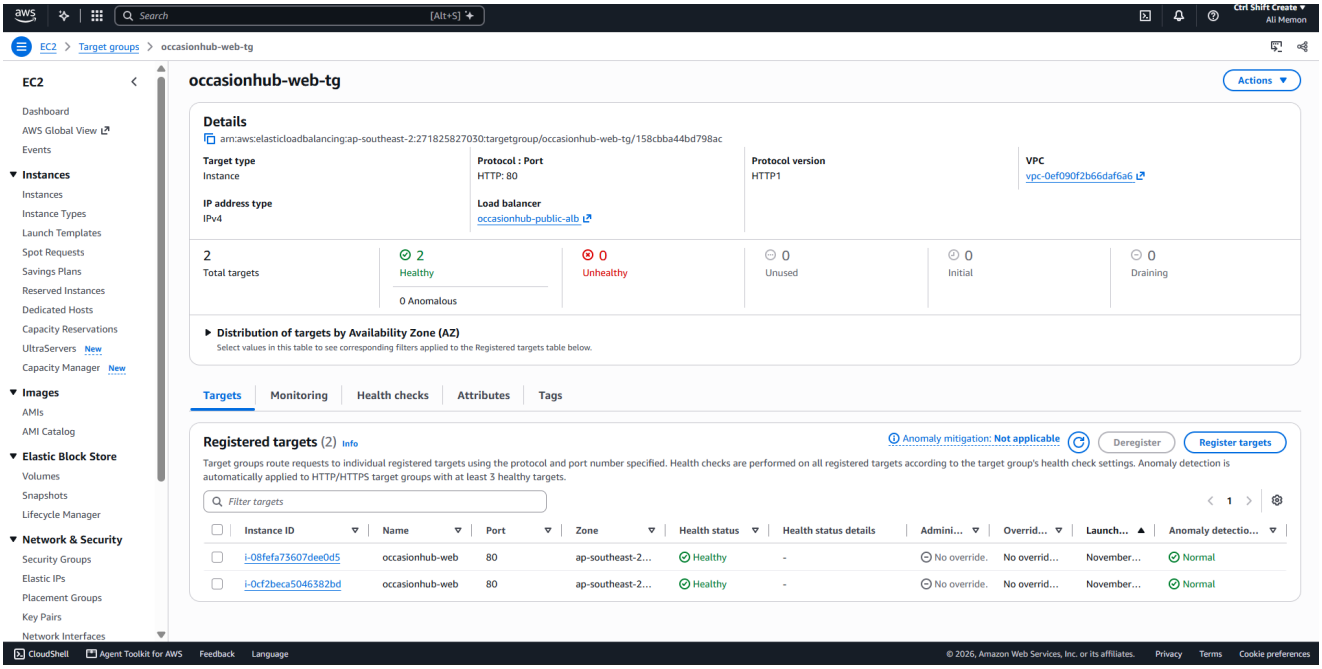


Figure 7: Target group `occasionhub-web-tg` behind the public ALB: both web instances healthy after the rolling deploy.

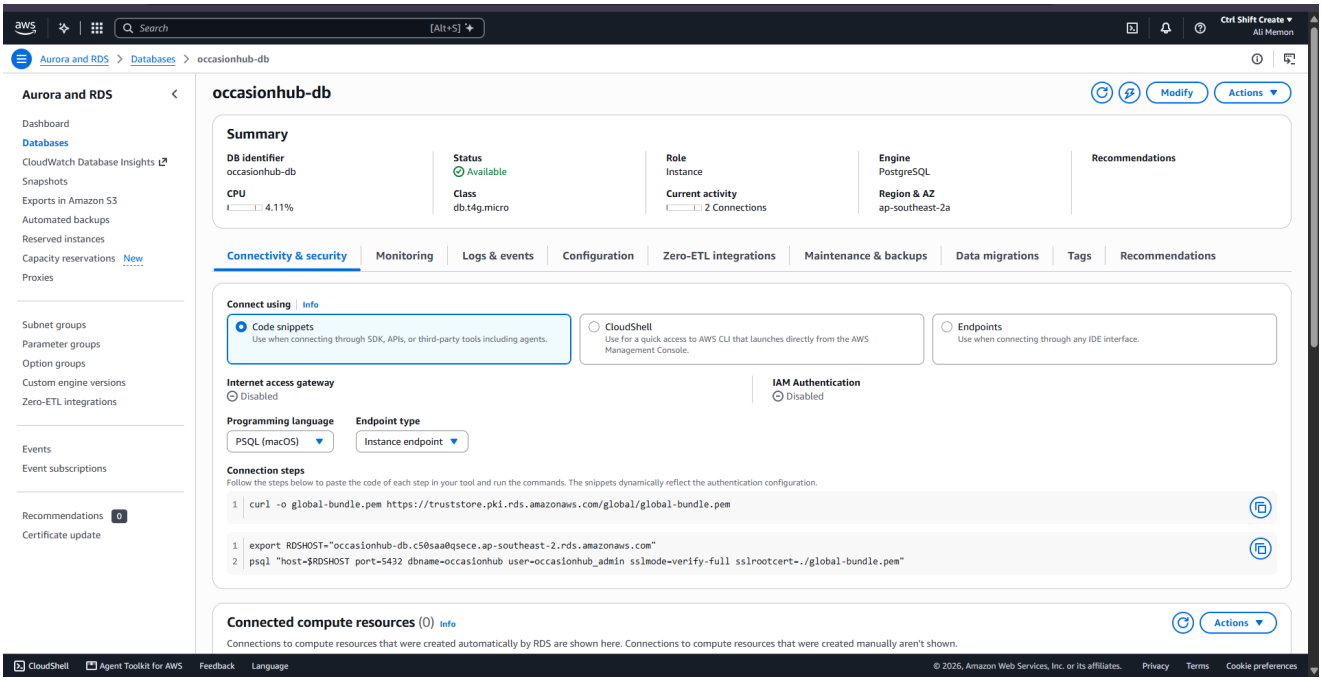


Figure 8: RDS instance `occasionhub-db`: PostgreSQL on db.t4g.micro, available, internet access gateway disabled.

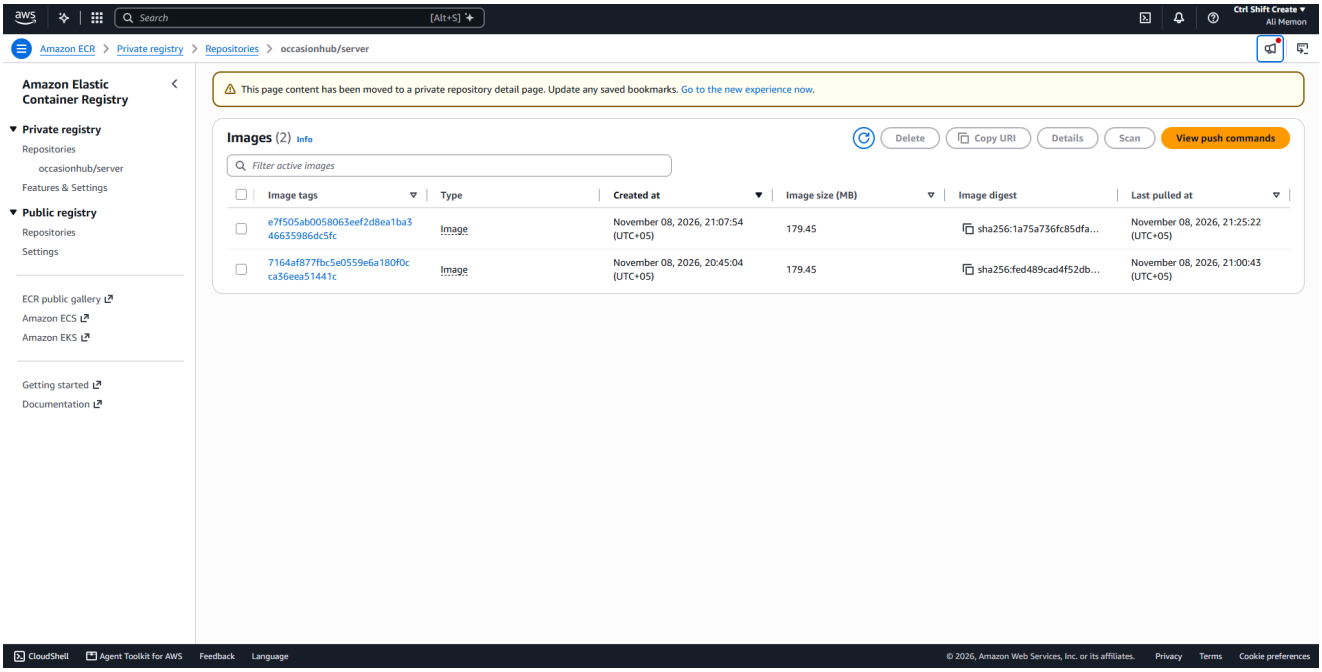


Figure 9: ECR repository `occasionhub/server`: one image per release, tagged with the git commit SHA (`7164af8`, `e7f505a`).

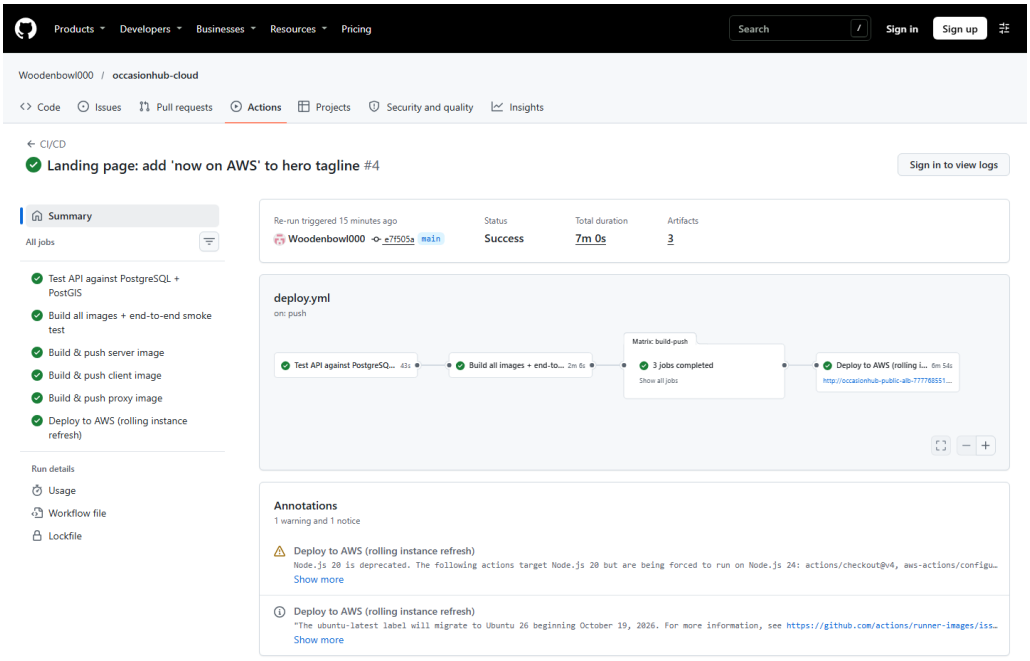


Figure 10: GitHub Actions run #4: every job green (the deploy job shown after its re-run).

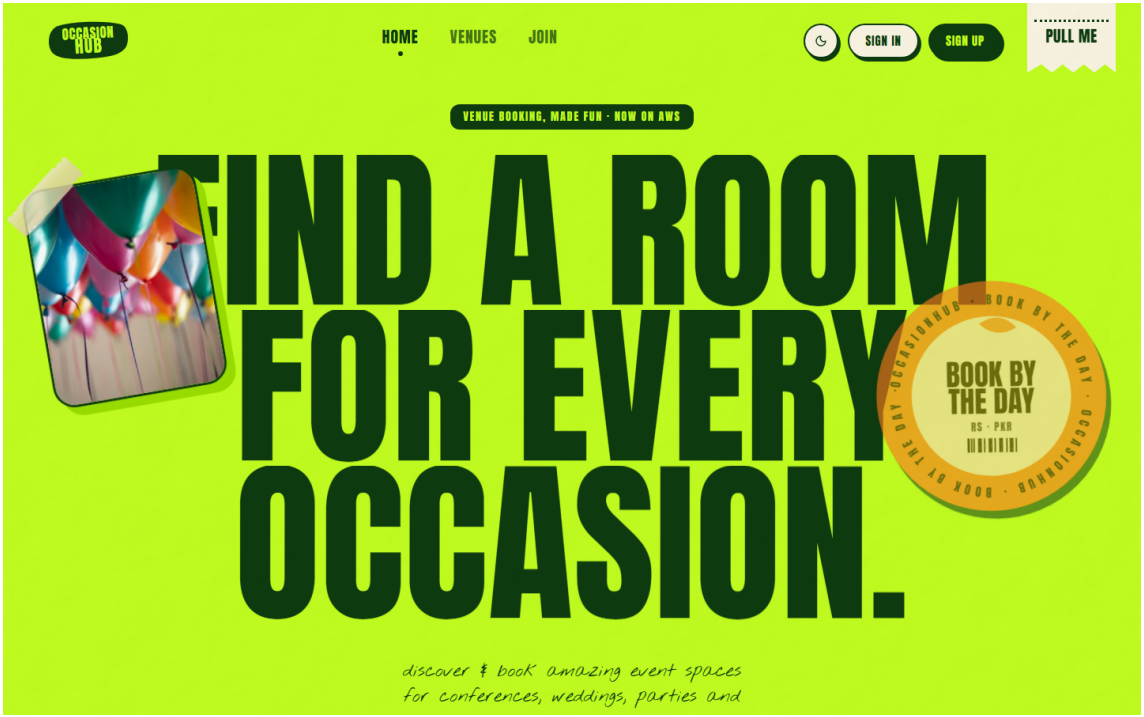


Figure 11: OccasionHub served through the public ALB after the automated deployment: the tagline now ends “now on AWS”.

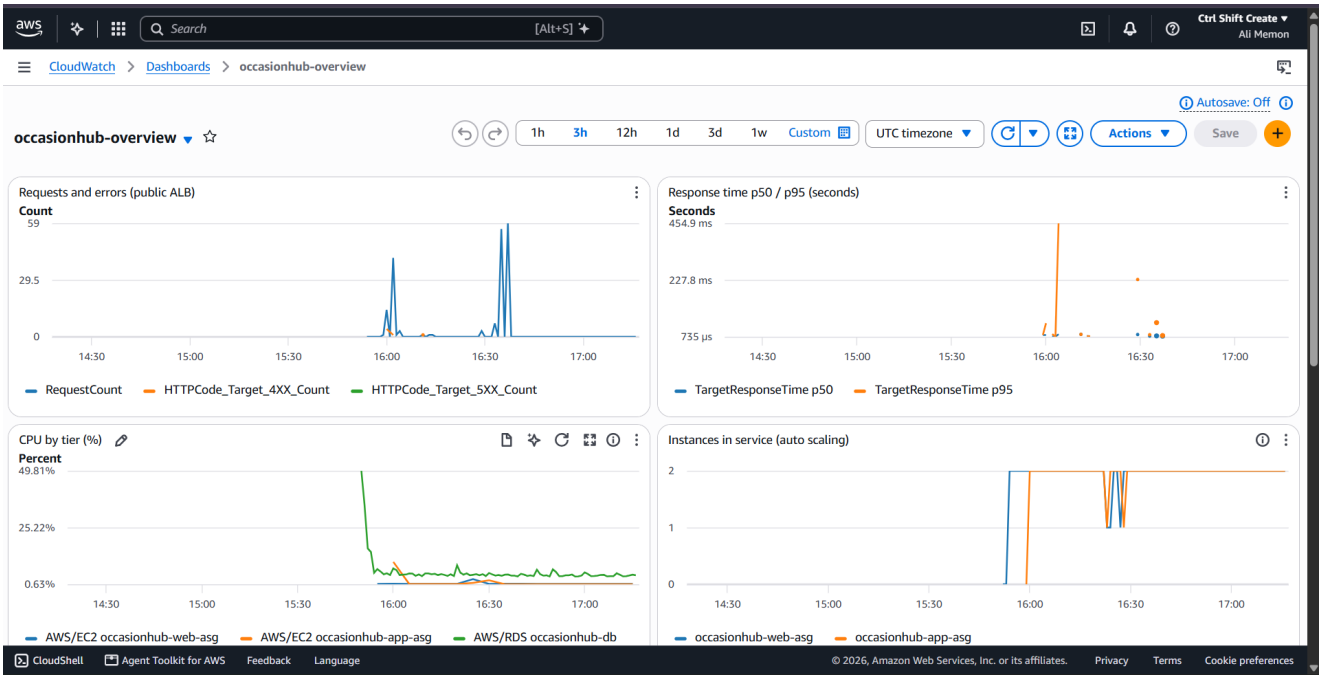


Figure 12: CloudWatch dashboard `occasionhub-overview`: requests, latency, CPU by tier and instances in service. The dips at 16:25 are the rolling deploy.

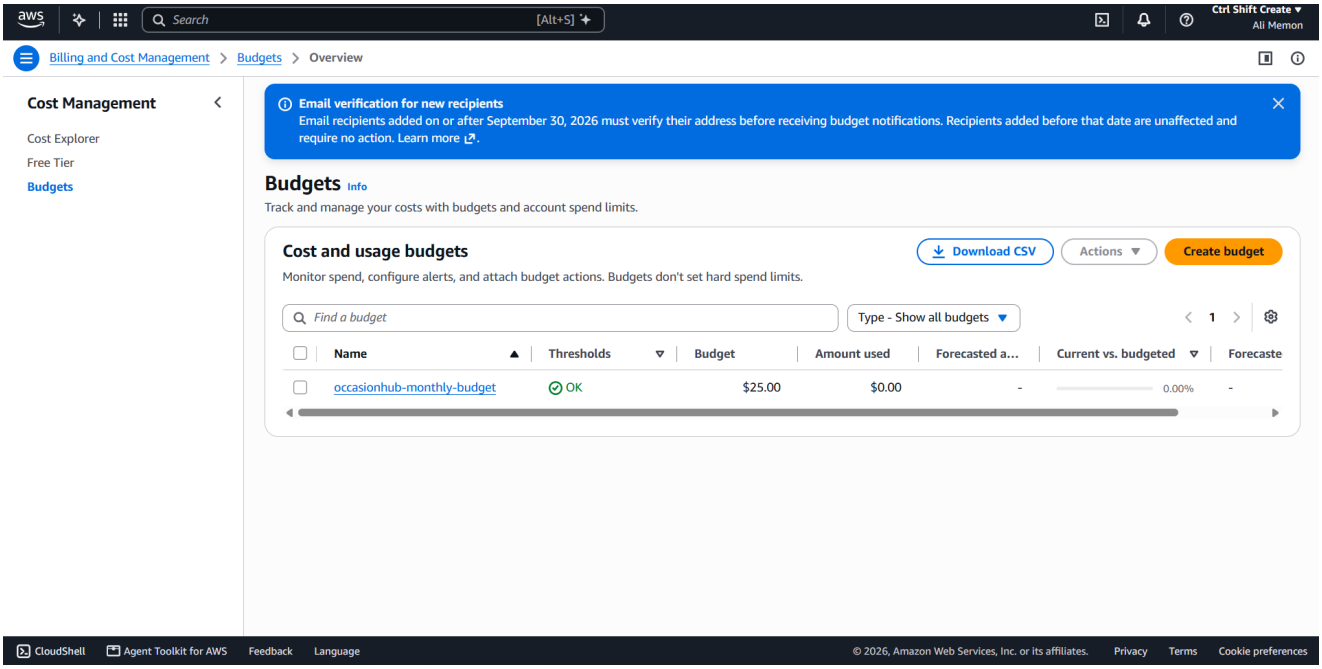


Figure 13: AWS Budgets: `occasionhub-monthly-budget` (USD 25 per month), status OK.

Local deployment (docker compose / local stack), captured on November 8, 2026:

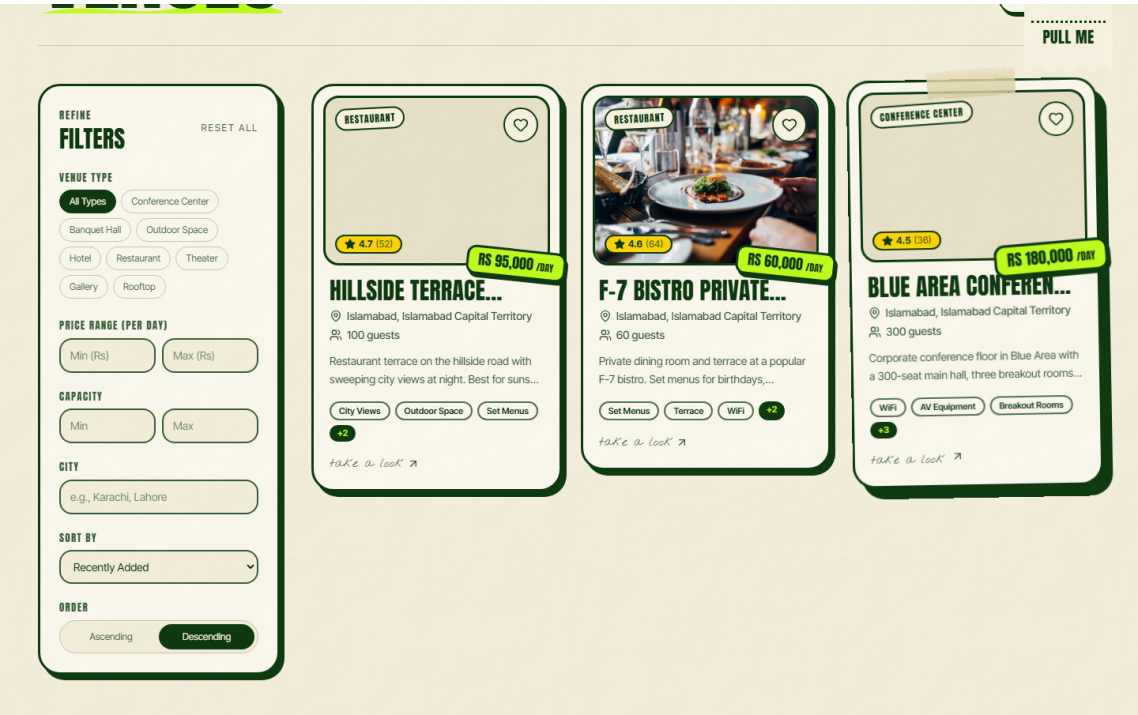


Figure 14: Venue search page served through the Nginx web tier, with data from the API and the PostGIS database.

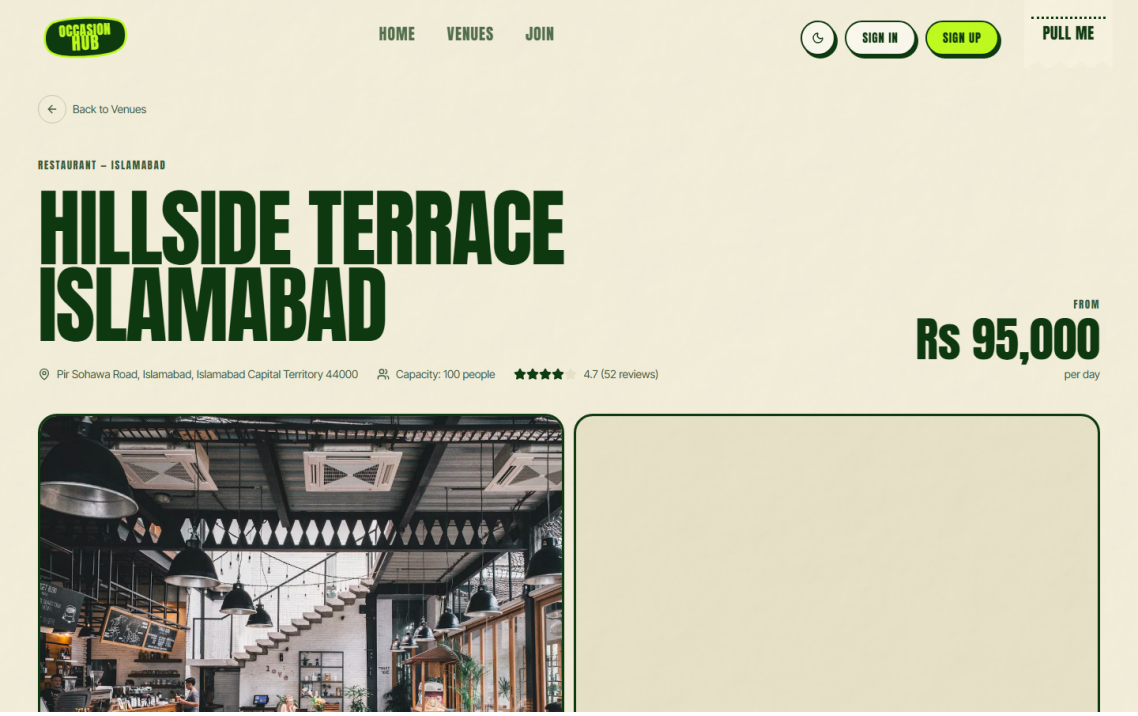


Figure 15: Venue detail page (front end, API and database working together).

Appendix B: Source files

Repository: `https://github.com/Woodenbowl000/occasionhub-cloud`. Main deliverable files:

- `server/Dockerfile`, `client/Dockerfile`, `proxy/Dockerfile`, `docker-compose.yml`
- `.github/workflows/deploy.yml` (CI/CD pipeline), `scripts/deploy.sh` (rolling deploy and rollback)
- `infra/terraform/*.tf` (network, security, data, registry, IAM, load balancers, compute, monitoring, outputs) and `templates/*.tftpl`
- `server/docker-entrypoint.js` (secrets loader and migrations), `server/tests/api.test.ts`
- `server/prisma/migrations/20261108000000_performance_indexes`

- `docs/architecture.svg`, `docs/benchmark.md`, `README.md` (deployment runbook)